

findGEVE.py

Main purpose of the findGEVE.py

Identify Giant Endogenous Viral Elements (GEVEs) in eukaryotic genome assemblies.

More detail about this script:

- Inspired from Viralrecall with many improvement for more robust GEVE identification.
- Fully written in Python with only one external dependency, which is BLAST+.
- Has automated TIR and TSD identification based on flanking region each candidate GEVE.
- Output files suitable for downstream analysis.
- Using 3 different hmm database: NCLD hallmark, GVOG, and Pfam v38.

How to install

This tool fully written in python and mostly use third party python package that can be install using pip. findGEVE.py is compatible with python v3.9 or hinger.

Third party Python packages

- numpy
- pandas
- pyfastx
- pyhmmer
- pyrodigal

External Python packages:

- blastn (from BLAST+)

Command:

```
pip install numpy pandas pyfastx pyhmmer pyrodigal  
conda install bioconda::blast
```

How to install

After all dependency already installed, type help command (`python findGEVE.py -h`) to see if this tool work well.

```
findGEVE.py - Identify Giant Endogenous Viral Elements (GEVEs) in eukaryotic genome assemblies.
```

```
Usage: findGEVE.py -db <directory> --prefix <prefix> <genome.fa> [OPTIONS]
```

Mandatory:

<code>-db, --db</code>	HMM database directory (must contain NCLDV_markers.hmm and gvog.hmm; Pfam-A.hmm is optional)
<code>--prefix</code>	Output prefix for GEVE IDs and file names
<code>genome</code>	Input genome assembly FASTA (gzip is acceptable)

Optional:

<code>--outdir</code>	Output directory	[default: ./Result_<YYYYMMDD>]
<code>-t, --threads</code>	CPU threads for ORF prediction and HMM search	[default: 4]
<code>-e, --evaluate</code>	E-value cutoff for HMM searches	[default: 1e-5]
<code>--blastn-jobs</code>	Parallel TIR-detection workers	[default: --threads]
<code>--min-hallmark-type</code>	Minimum number of distinct hallmark types required per cluster	[default: 2]
<code>-h, --help</code>	Show this help and exit	

Download the database

This tool using 3 different hmm database, including NCLDV hallmark, GVOG, and Pfam.

- For NCLDV_markers.hmm already available in this repository database/NCLDV_markers.hmm. However, thanks to Dr. Frank Aylward and his team who made this database.
- Giant Virus Orthologs Groups database (GVOG) can be downloaded from their official website <https://faylward.github.io/GVDB/>. Make sure to download the .hmm file and rename it into gvog.hmm. Or you can simply use this command to download:

```
wget https://zenodo.org/record/4728209/files/GVOGs.tar.gz
```

- Pfam database latest version can be downloaded from its official website <https://www.ebi.ac.uk/interpro/download/pfam/> or by using this command:

```
wget https://ftp.ebi.ac.uk/pub/databases/Pfam/current_release/Pfam-A.hmm.gz
```

Then unzip the database files and put the .hmm files into same folder with NCLDV_markers.hmm

database/

├─ NCLDV_markers.hmm

├─ gvog.hmm

└─ Pfam-A.hmm

How this tool works

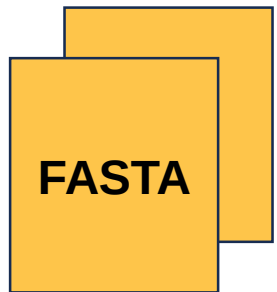
This tool has 6 main stage for robust GEVE identification. It can be use for contig or chromosome level genome assemblies

Step by step findGEVE.py

1. Gene prediction using pyrodigal (ignore contig length ≤ 50 kb)
2. Hallmark genes scanning and retain contig with positive hallmark genes. Then, GVOG scan and Pfam scan on contig with positive hallmark genes.
3. Seeding candidate GEVE by sliding window over hallmarks and cluster region with ≥ 2 hallmark types within 300 kb.
4. Calculate rolling viral score per ORF (window=15) = smoothed virbit – pfambit signal to decide candidate GEVE region.
5. Per-cluster boundary refinement by TIR detection. TIR detection using blastn self vs self candidate GEVE for boundary refinement. If TIR not found, GEVE boundary will be decided based on rolling viral score.
6. TSD identification in case found candidate GEVE with TIR.

1. Gene prediction

Prokaryotic gene prediction by pyrodigal in metagenomic mode (contig ≥ 50 kb)



Genome sequence

Input fasta file can be gzipped.
It works well for contig and
chromosome level genome.



pyfastx

Indexing by pyfastx

Index the fasta file. pyfastx also
help to read gzipped fasta file.



Pyrodigal

Star 184

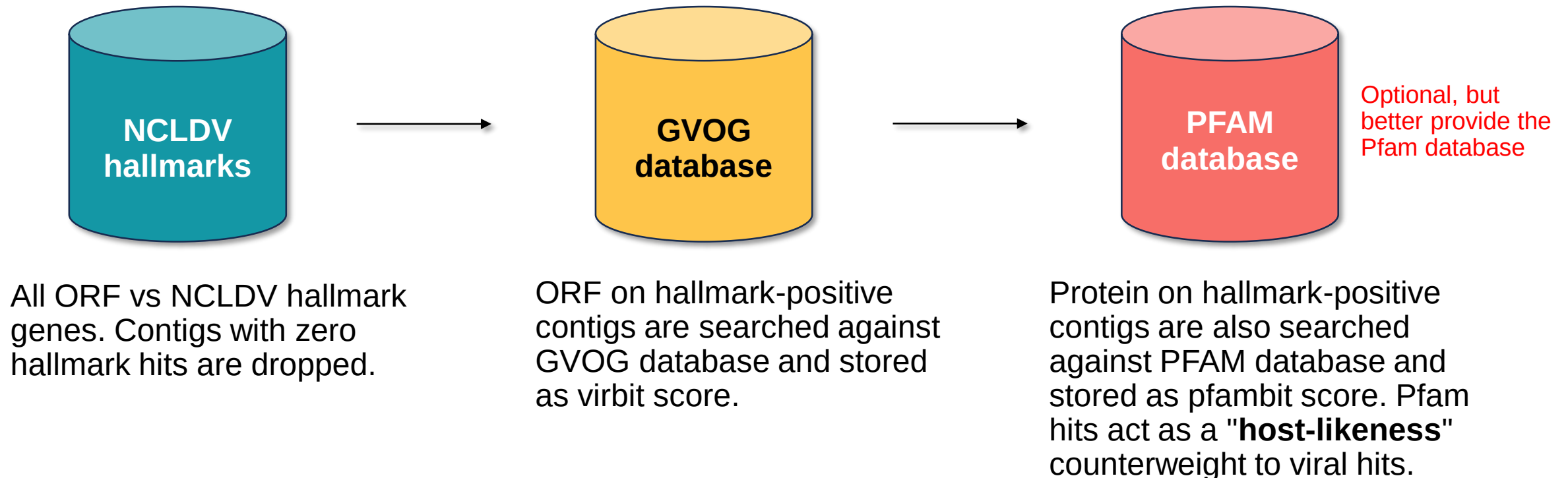
Prokaryotic gene prediction

Filtered contig (≥ 50 kb) will be use
as input for pyrodigal and resulting
protein sequence. pyrodigal will be
run in metagenomic mode

2. HMM annotation

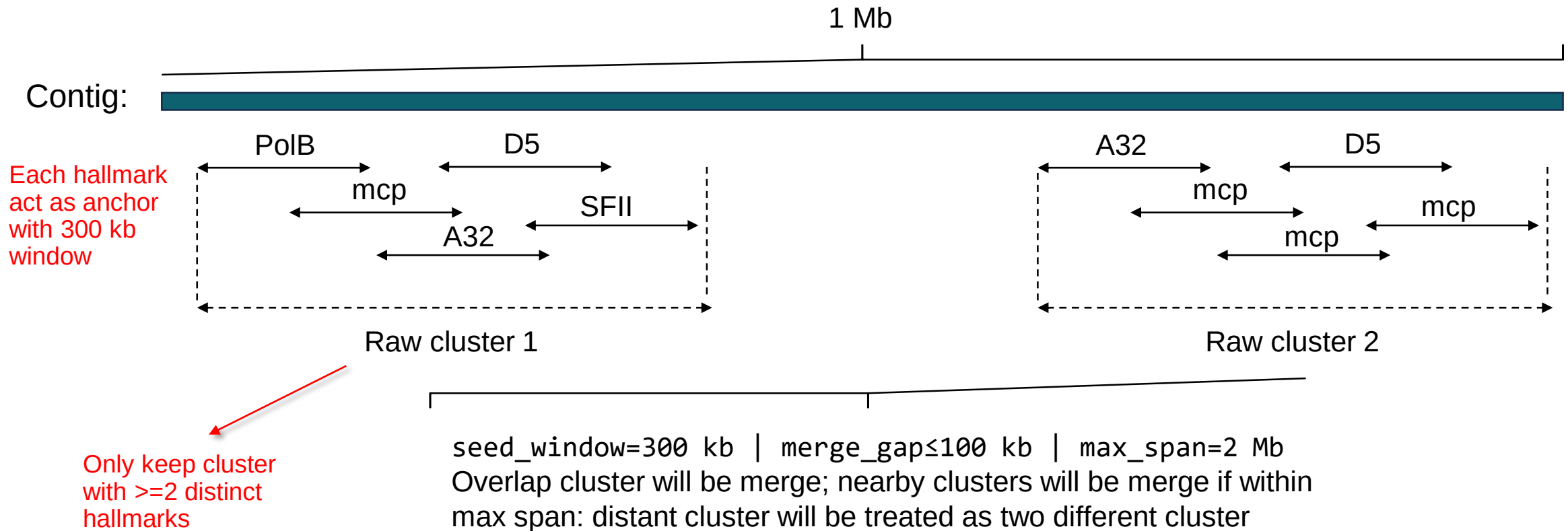
Three sequential pyhmmer scans, narrowing the search space at each step.

List of hallmark genes: **A32, D5, SFII, mcp, mRNAC, PolB, RNAPL, RNAPS, RNR, VLTF3**



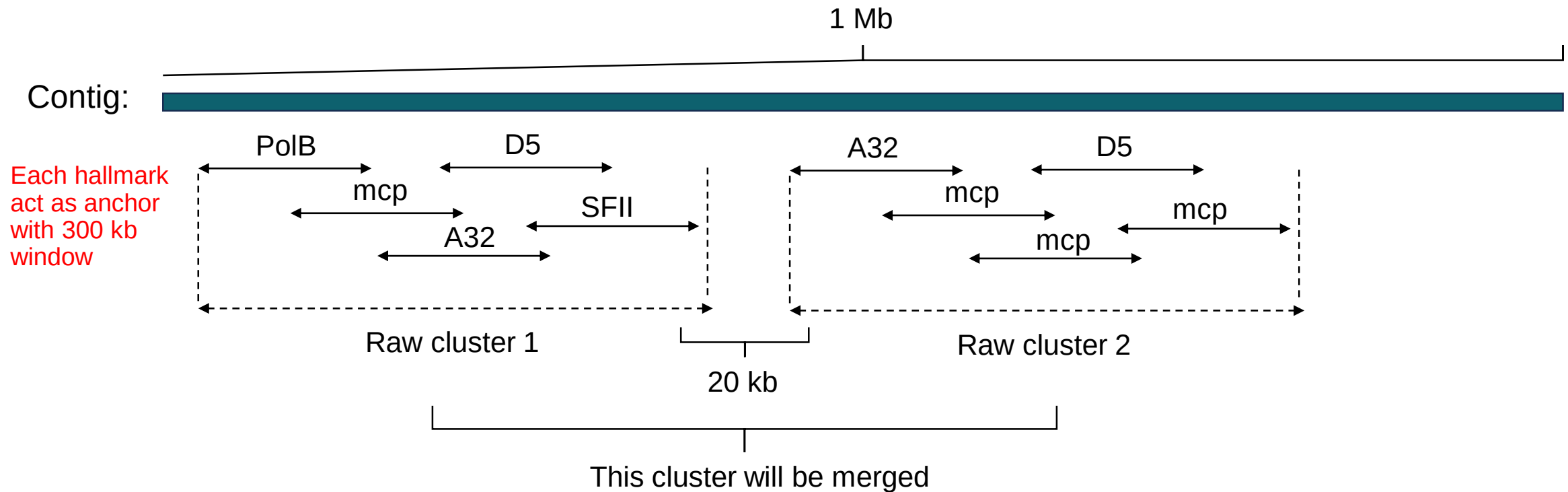
3. Seed candidate GEVE via hallmark clustering

Slide a 300 kb window over hallmark genes and require ≥ 2 distinct hallmark types.



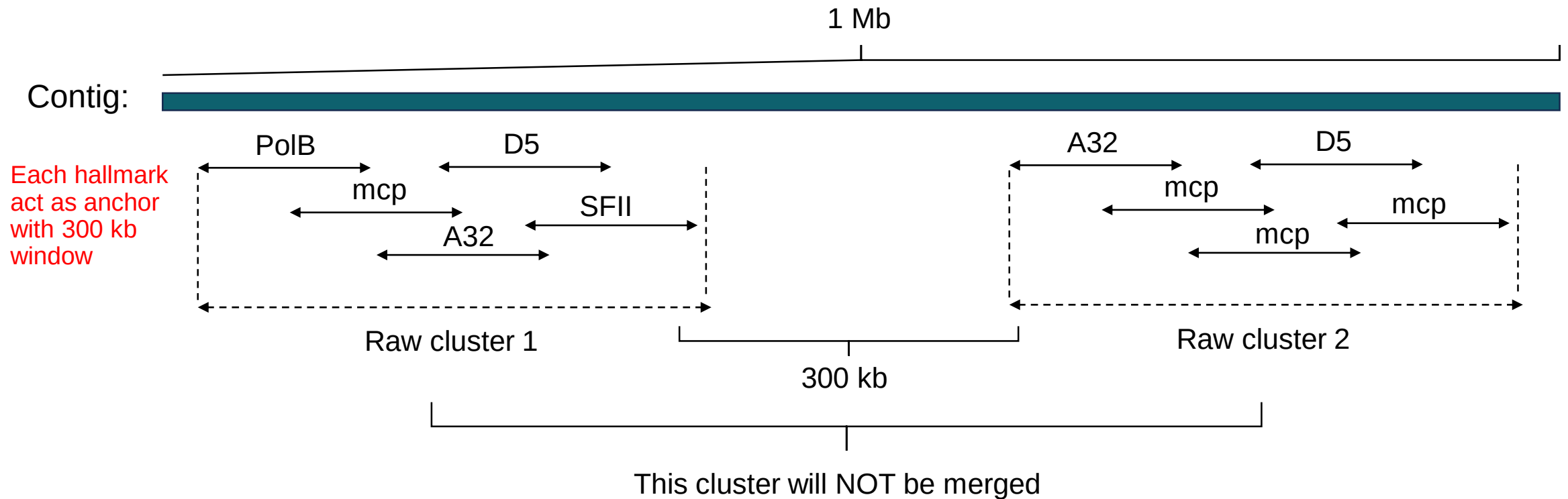
3. Seed candidate GEVE via hallmark clustering

Slide a 300 kb window over hallmark genes and require ≥ 2 distinct hallmark types.



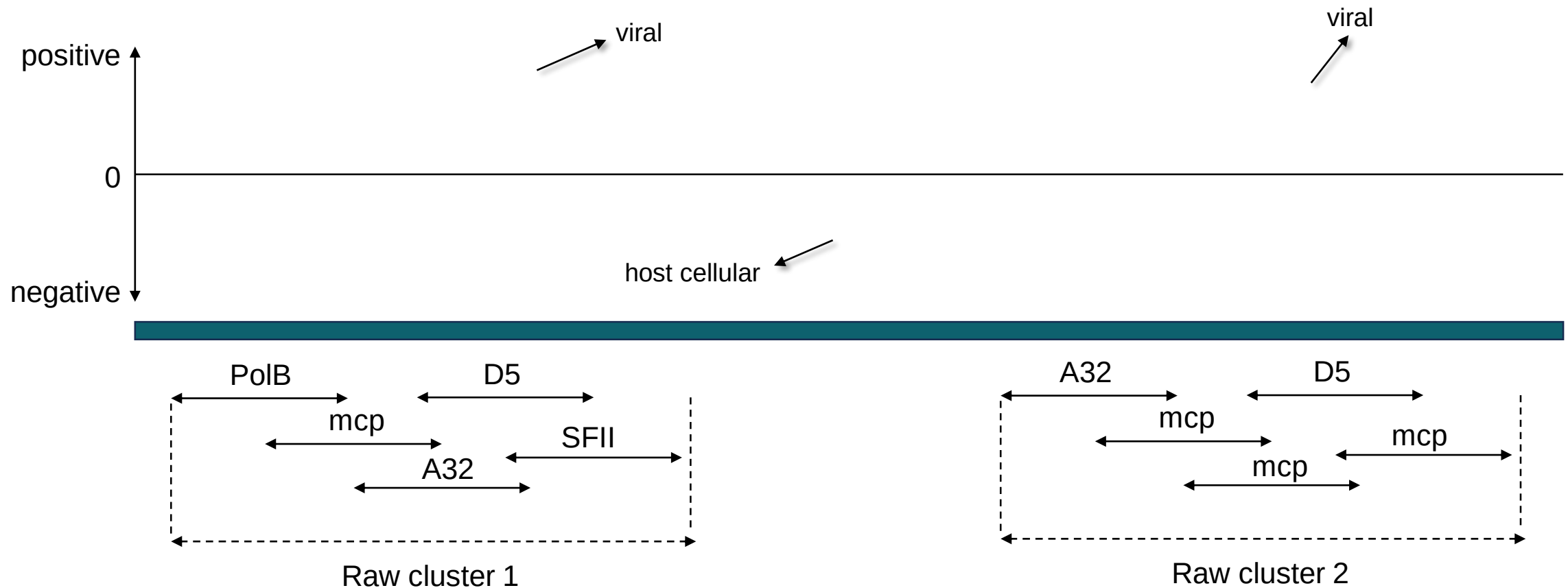
3. Seed candidate GEVE via hallmark clustering

Slide a 300 kb window over hallmark genes and require ≥ 2 distinct hallmark types.



4. Rolling viral score calculation

Per candidate contig Viral score = virbit - pfambit to retain high confident viral region.



About viral score

- **Giant Virus Orthologous Groups (GVOG)** database covering most NCLDV-encoded functions.
- **Pfam** database is dominated by cellular protein families.

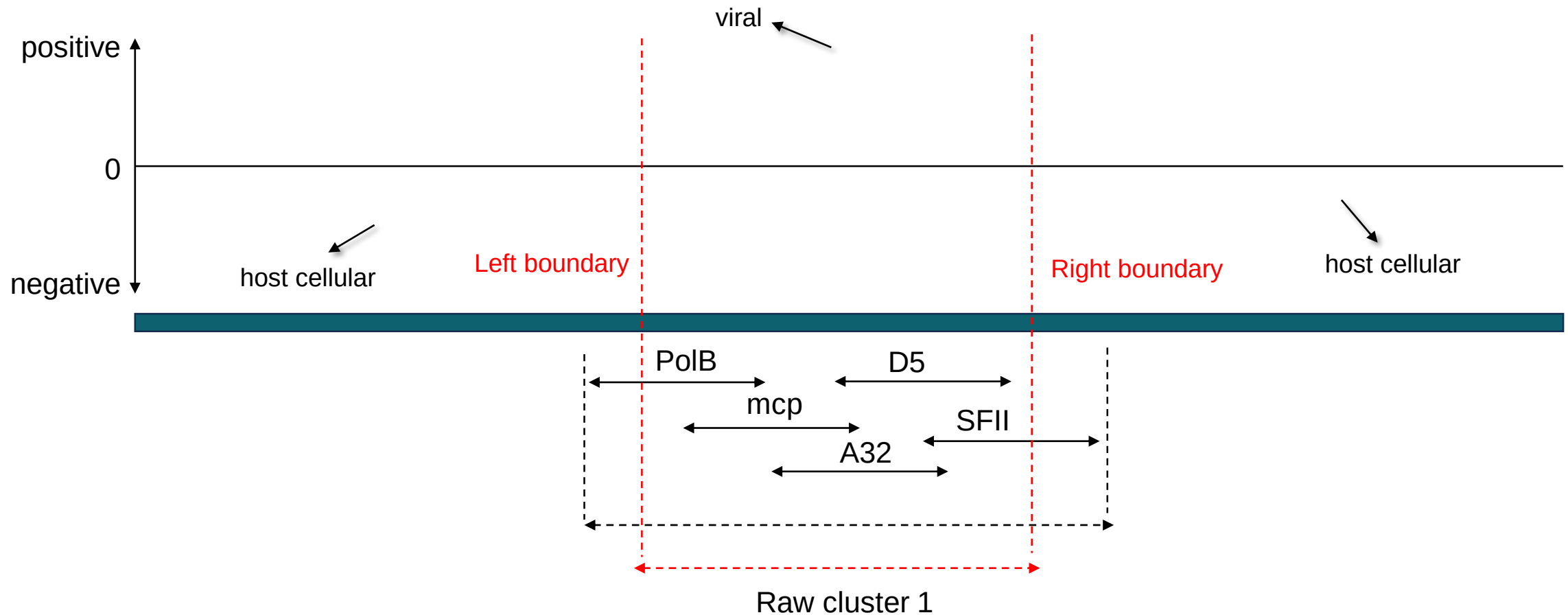
A dark-themed terminal window with three colored window control buttons (red, yellow, green) in the top-left corner. It displays the formula for calculating the viral score.
$$\text{viral score} = \text{virbit} - \text{pfambit}$$

Then viral score calculated by subtracting virbut (GVOG) with pfambit (PFAM). Viral region will be resulting in positive value and host likeness (cellular) will resulting zero or negative value.

*“So, a **strong Pfam hit** on an ORF without a **strong viral hit** suggests the protein (ORF) is part of the host, not the integrated viral element.”*

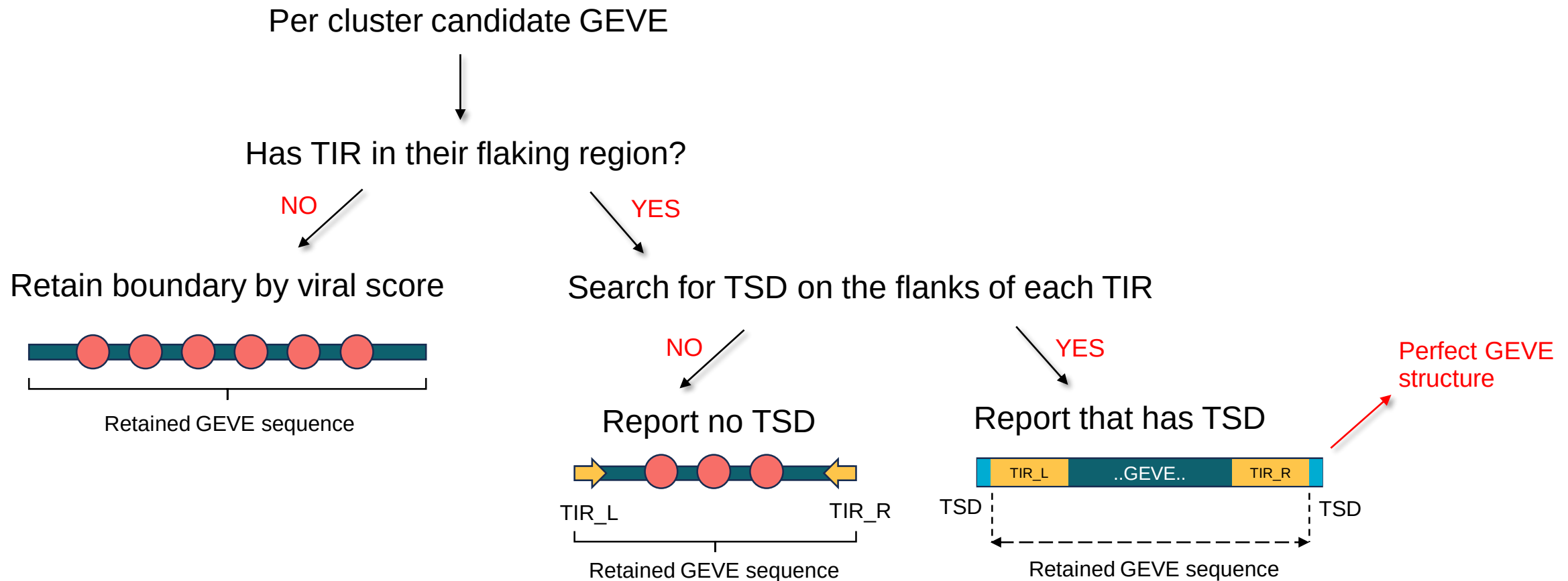
4. Rolling viral score calculation

Per candidate contig Viral score = virbit - pfambit to retain high confident viral region.



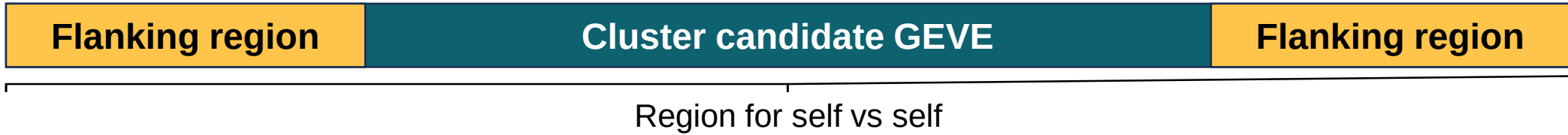
5. Per-cluster boundary refinement

Each cluster candidate GEVE is processed end-to-end to retain its robust region.

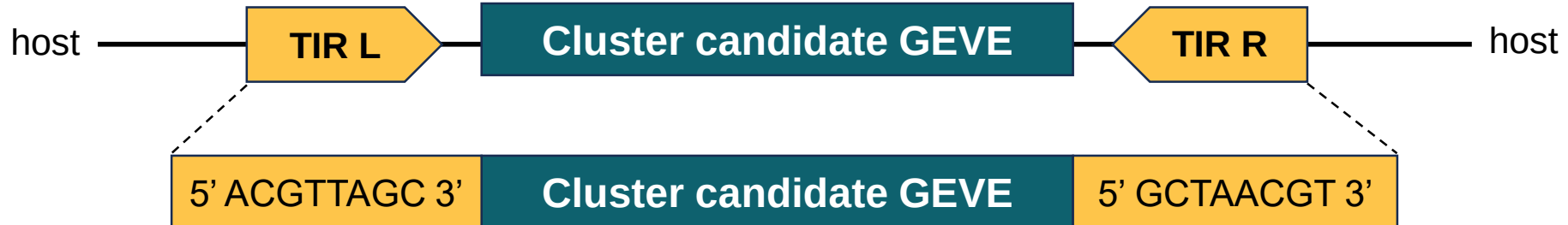


TIR detection algorithm

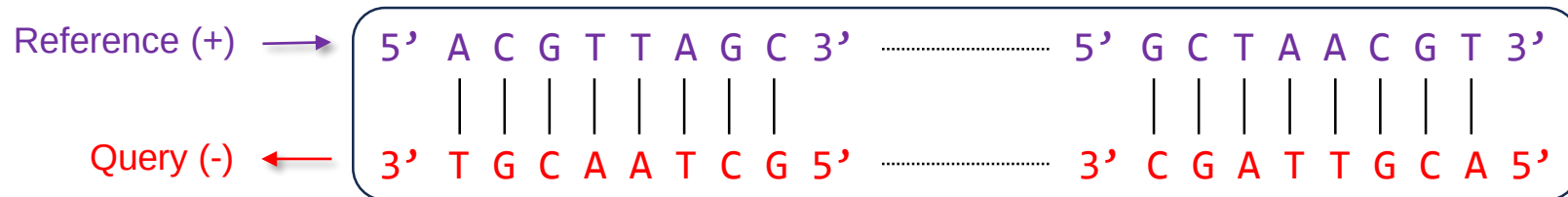
Self-vs-self blastn forward with reverse-complement strand on the cluster candidate GEVE $\pm$ flanking.



Suppose we have GEVE with TIR in it flanking region:



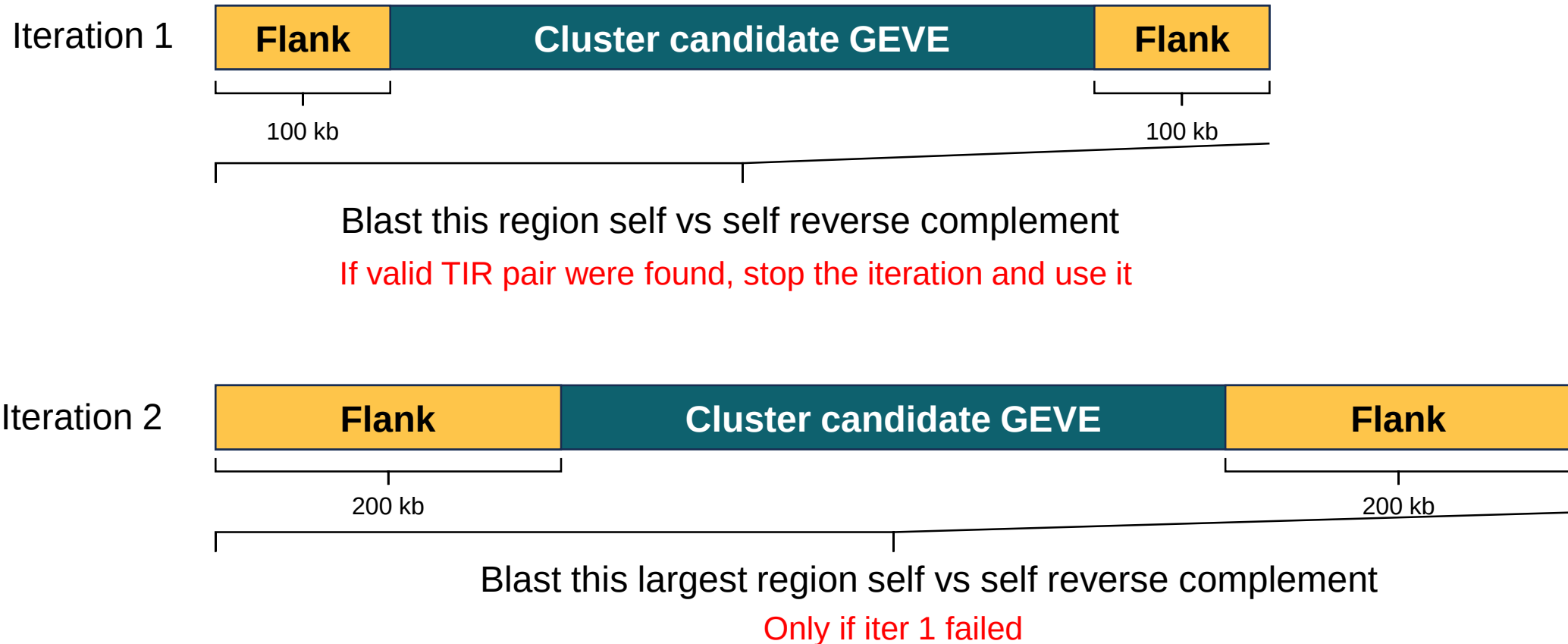
Run BLAST: reference = region (plus strand) vs query = region (minus strand, reverse-complemented)



These two segments align perfectly → one BLAST hit = one TIR pair

TIR detection algorithm

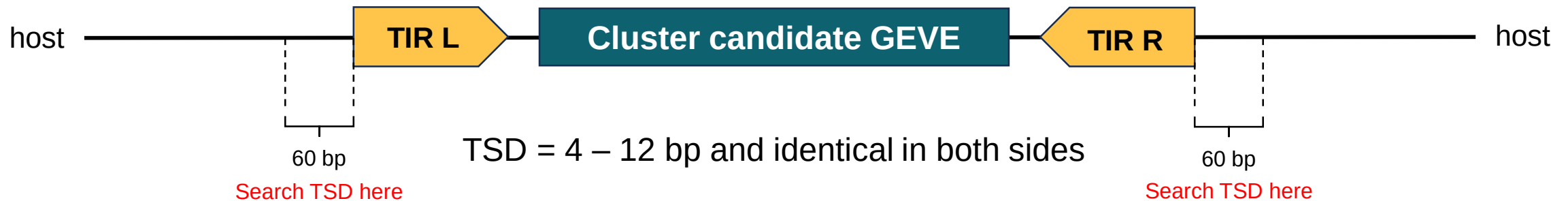
We don't know how big the GEVE is yet, so we don't know how wide a window to BLAST.
findGEVE.py tries growing windows (a "flank ladder") to find a TIR sequence:



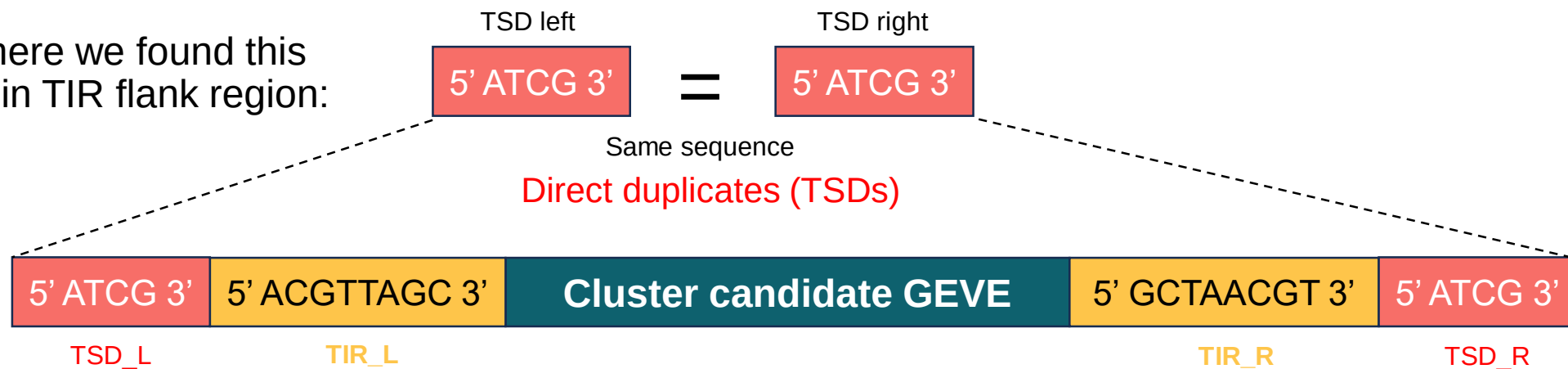
**length scale in this illustration is not calibrated!*

6. TSD detection algorithm

Many GEVE duplicate a **short host sequence** (TSD) on both flanks of the insertion. So, if TIR pair were found, then check the target site duplication (TSD) in TIR flank region.



Suppose here we found this sequence in TIR flank region:



Output files

Retained GEVE are sorted by contig and position, assigned IDs, and store to output directory.

```
— <prefix>.geve.fna      # Retained GEVE nucleotide sequences
— <prefix>.summary.tsv   # Per-GEVE summary table (length, hallmark, start, end, etc.)
— <prefix>.markerout     # Hallmark, TIR, and TSD coordinate for each GEVE
— <prefix>.geve.gff3     # Annotatated GEVE relative to host genome
— <prefix>.geve.cds      # Per-GEVE coding sequence
— <prefix>.geve.pep      # Per-GEVE protein sequence
— hallmark/             # Per-hallmark type protein sequence for all retained GEVE
  — <prefix>.a32.pep
  — <prefix>.mcp.pep
  — <prefix>.hallmark.pep
```